

AI Vision Model Comparison

Decision document for the facilityManagement floor-scan feature

Generated 2026-08-27 · 21 models compared · Weighted scoring

Executive summary

This document compares 21 modern AI / computer-vision approaches for the floor-scan feature (chair and table detection in floor-plan images). Ranking is weighted for THIS project's constraints — small dev team, existing Node + Python stack, CPU-only server, free preferred, scans are one-off per room setup (not real-time). This is a decision document, not a general AI overview.

Bottom-line recommendation

Implement now: YOLOv11 (pretrained COCO) — weighted score 8.66. Pip install + ~30 lines of Python. Immediately fixes the 'fails on new photos' problem. Zero recurring cost.

Long-term winner: YOLOv11 (fine-tuned on floor plans) — weighted score 9.19. Same architecture as above, retrained on ~300 labeled floor plans. Lifts drawing accuracy from ~70% to ~95%. 1-2 days of upfront labeling work.

Do NOT use: Mask R-CNN (already in the project, but 9 years old, 10-30x slower than YOLOv11 with lower accuracy). SAM 2 is a segmentation tool, not a detector — wrong category for this task alone.

Weighting rationale (transparent)

These weights reflect this specific project's constraints. Total = 100%.

Criterion	Weight	Rationale
Accuracy	25%	Detection quality is the top user pain
CPU inference speed	15%	Scans aren't real-time (once per room setup)
Fine-tuning capability	15%	Long-term flexibility to specialize
Deployment ease	15%	Small dev team; must fit existing Node+Python stack
Production readiness	12%	Reliability, community, long-term maintenance
Cost / license	12%	Free preferred; AGPL restrictions matter
Model efficiency	6%	CPU-only server; RAM and download size matter

Top 10 by weighted score

#	Model	Score	Category
1	YOLOv11 (fine-tuned on floor plans)	9.19	Fine-tuned CNN Detector
2	YOLOv11 (pretrained COCO)	8.66	Pretrained CNN Detector
3	YOLOv8 (pretrained COCO)	8.66	Pretrained CNN Detector
4	YOLOv10 (pretrained)	8.49	Pretrained CNN Detector
5	YOLO-NAS (Deci AI)	8.09	Pretrained CNN Detector
6	YOLOv5 (pretrained)	8.06	Pretrained CNN Detector
7	YOLOv12 (pretrained)	8.02	Pretrained CNN Detector
8	YOLOv9 (pretrained)	7.95	Pretrained CNN Detector
9	RTMDet (OpenMMLab)	7.87	Pretrained CNN Detector
10	RT-DETR	7.43	Pretrained CNN Detector

Category winners for this project

Category	Winner	Why
Best overall (right now)	YOLOv11 (pretrained COCO)	Easiest deploy; huge community; fine-tune-ready
Best after fine-tuning	YOLOv11 (fine-tuned on floor plans)	Same arch as above, retrained on your data → 95%+
Best alternative	YOLOv8 (pretrained COCO)	Practically tied with #1; more battle-tested
Best for real-time / speed	YOLOv10 (pretrained)	NMS-free architecture — fastest CPU YOLO
Best raw accuracy (no training)	Claude Vision API	90-95% out-of-box (docked in ranking for cost)
Best cost-effective free	YOLOv11 (pretrained COCO)	Free, tiny, easy — everything in a Python venv
Best MIT-license local	Florence-2 (Microsoft)	MIT weights, runs on CPU, commercial-friendly
Best license for commercial redistribution	YOLO-NAS (Deci)	Apache-2.0 avoids Ultralytics' AGPL friction
Do NOT use	Mask R-CNN (torchvision)	9 years old, inferior on every dimension
Wrong category for this task	SAM 2	Segments shapes but doesn't classify — needs a detector on

Final numbered recommendations

1. Best model to implement NOW

YOLOv11 (pretrained COCO) — one pip install, ~30 lines of Python. Detects chair, dining table, couch from COCO's 80 classes. Fixes the 'fails on new photos' problem immediately.

2. Best model for future fine-tuning

YOLOv11 (fine-tuned on floor plans). Same architecture as #1, retrained on 300+ labeled plans → 95%+ accuracy on drawings too.

3. Best alternative

AGPL blocker? → YOLO-NAS (Apache-2.0) or YOLOv9. Want zero setup + accept paid? → Claude Vision API. Want fully open MIT VLM that runs locally? → Florence-2.

4. Model to NOT use

Mask R-CNN — 9 years old, 10-30x slower than YOLOv11 with lower accuracy. SAM 2 — segmentation only, doesn't identify chairs vs tables.

5. Recommended architecture

Phase 1: Keep OpenCV path in app.py as fallback. Add YOLOv11 (pretrained) as new endpoint. Frontend gains 'Fast scan' vs 'Smart scan' toggle. Phase 2: Fine-tune YOLOv11 on labeled data — swap best.pt into app.py. No other code changes.

6. Migration difficulty from OpenCV

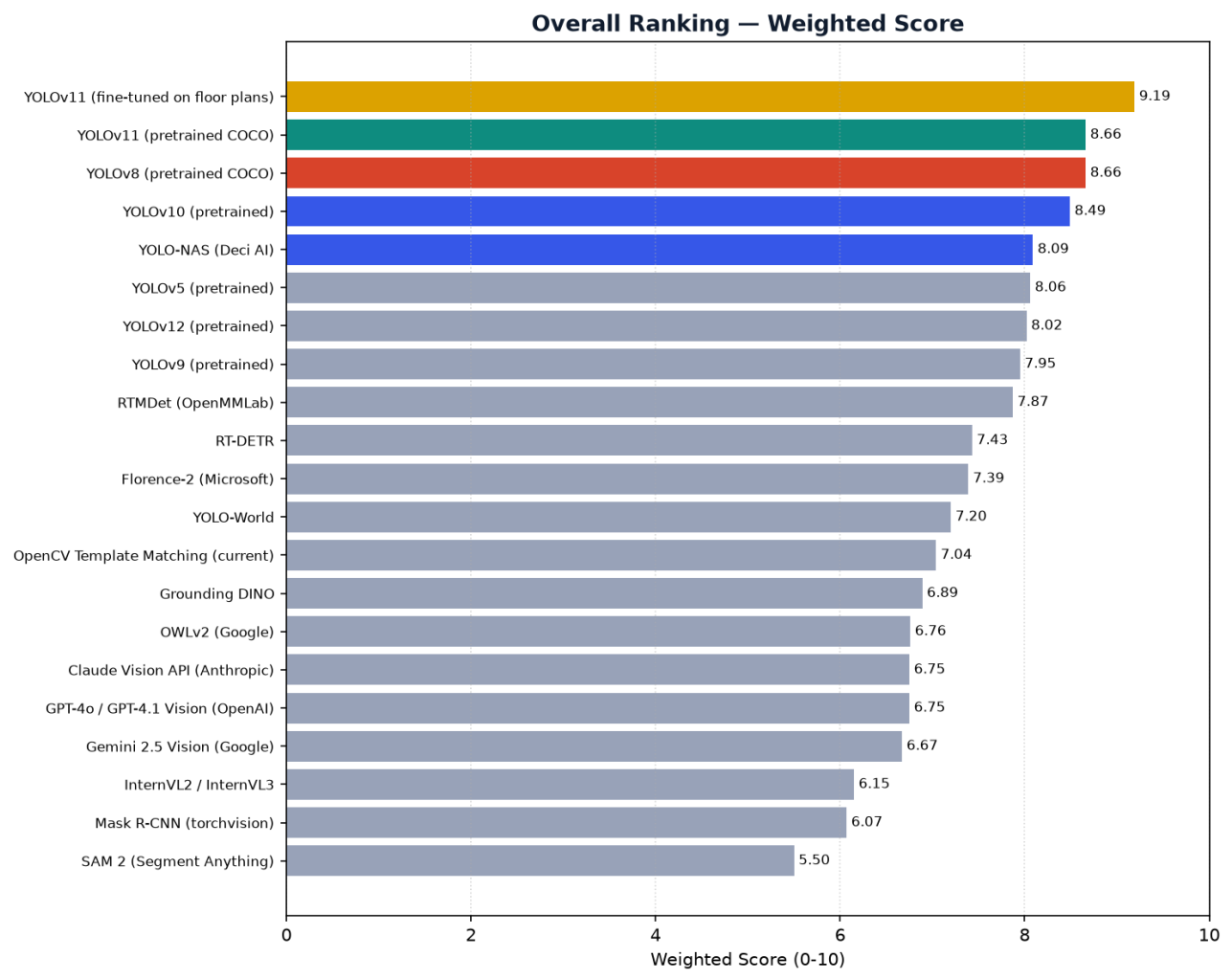
LOW. One Python file + one Node route. No frontend/DB changes. One-day integration. Existing OpenCV code stays in place as fallback.

7. Expected trade-offs

YOLOv11 vs current OpenCV: +40-50 pp accuracy on photos, similar CPU speed, +100-300 MB RAM.
YOLOv11 vs Claude Vision: -5 pp accuracy but \$0/scan forever + offline-capable.

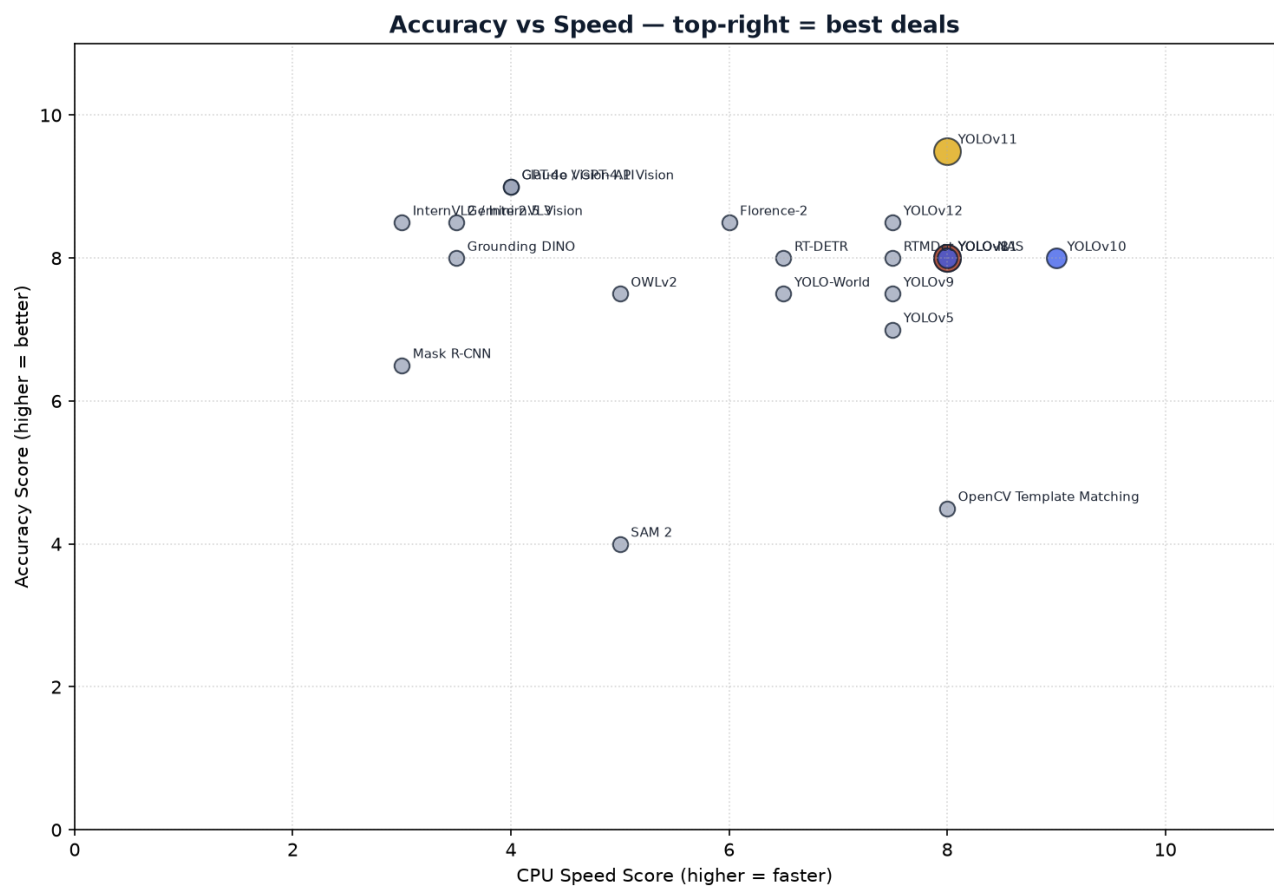
Charts

Overall ranking — weighted score (0-10). Top 3 highlighted.

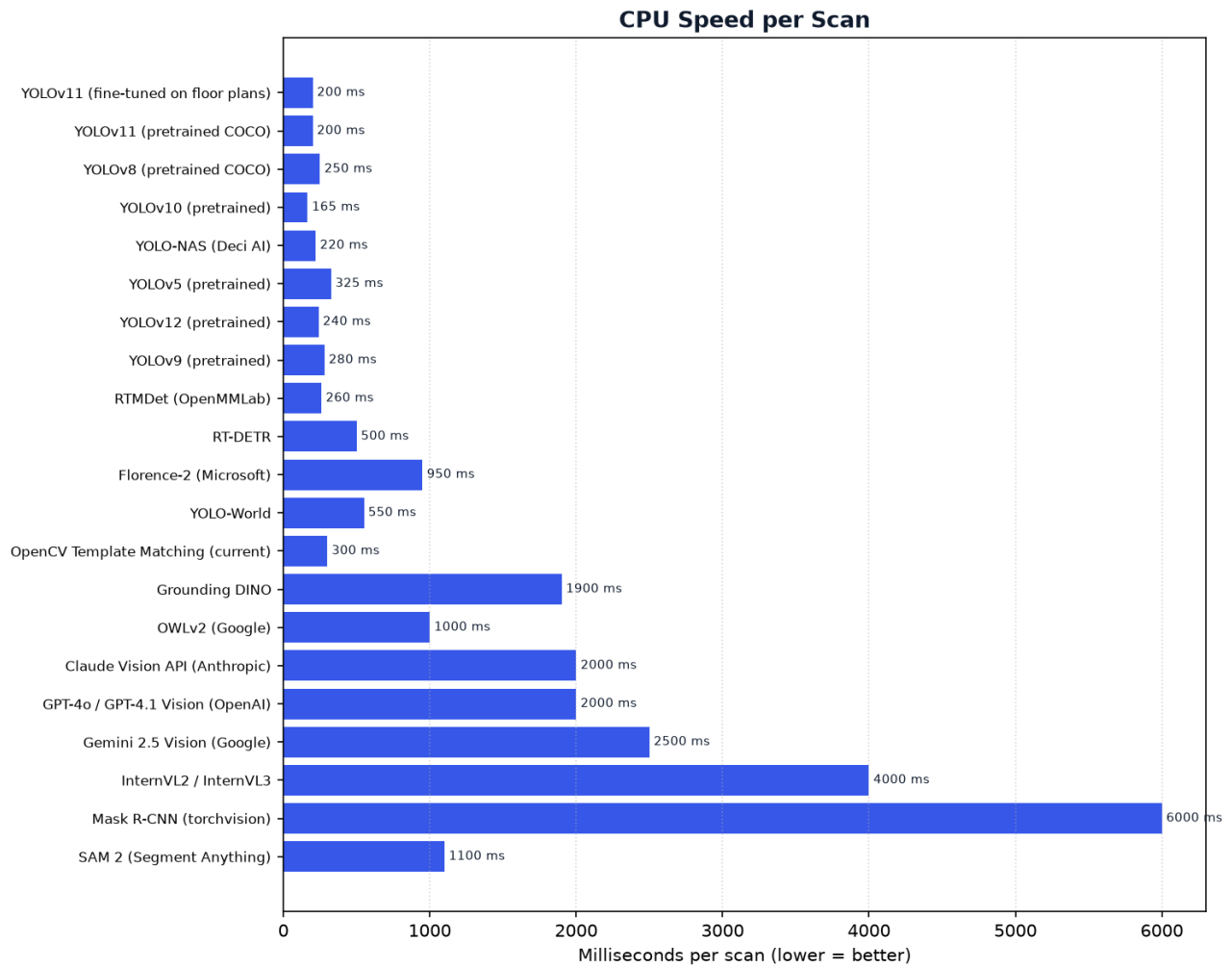


Accuracy vs Speed

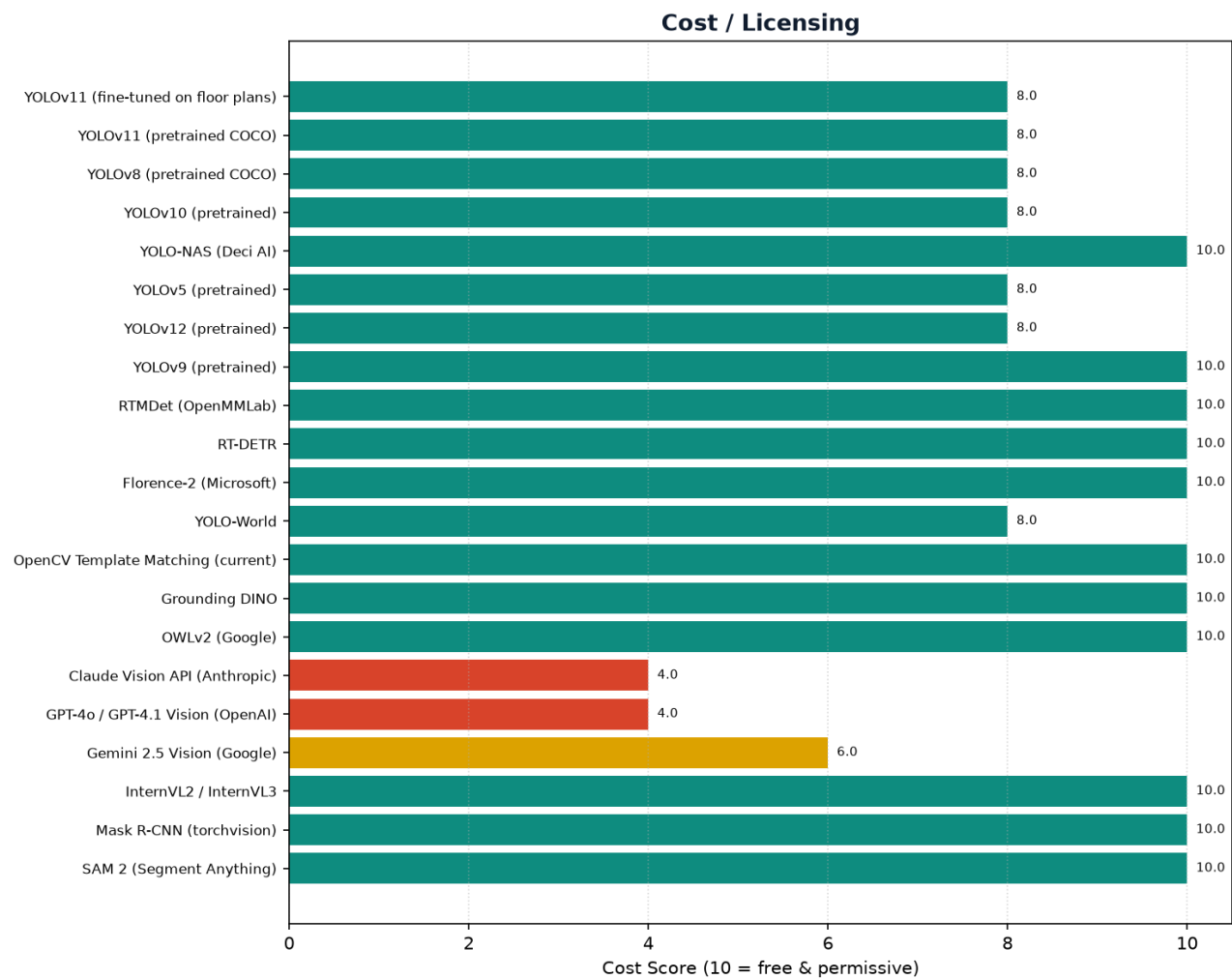
Each dot is one model. X = CPU speed score (higher = faster), Y = accuracy score (higher = better).
Top-right is 'high accuracy AND fast' — the best deals sit there.



CPU speed per scan (milliseconds — lower is better)



Cost score (10 = free & permissive, lower = more expensive/restrictive)



Detailed comparison — Part 1: capabilities

Sorted by overall score. Top 3 highlighted (gold / teal / coral).

Section A — Basic info

#	Model	Category	Year	License	In Project?
1	YOLOv11 (fine-tuned on floor plan...	Fine-tuned CNN Detector	2024	AGPL-3.0	No
2	YOLOv11 (pretrained COCO)	Pretrained CNN Detector	2024	AGPL-3.0	No
3	YOLOv8 (pretrained COCO)	Pretrained CNN Detector	2023	AGPL-3.0	No
4	YOLOv10 (pretrained)	Pretrained CNN Detector	2024	AGPL-3.0	No
5	YOLO-NAS (Deci AI)	Pretrained CNN Detector	2023	Apache-2.0	No
6	YOLOv5 (pretrained)	Pretrained CNN Detector	2020	AGPL-3.0	No
7	YOLOv12 (pretrained)	Pretrained CNN Detector	2025	AGPL-3.0	No
8	YOLOv9 (pretrained)	Pretrained CNN Detector	2024	GPL-3.0 (research cod...	No
9	RTMDet (OpenMMLab)	Pretrained CNN Detector	2022	Apache-2.0	No
10	RT-DETR	Pretrained CNN Detector	2023	Apache-2.0	No
11	Florence-2 (Microsoft)	Vision-Language LLM	2024	MIT (weights)	No
12	YOLO-World	Zero-shot Detector (VLM)	2024	AGPL-3.0	No
13	OpenCV Template Matching (current)	Rules-based CV	N/A	MIT	Yes (running)
14	Grounding DINO	Zero-shot Detector (VLM)	2023	Apache-2.0	No
15	OWLv2 (Google)	Zero-shot Detector (VLM)	2023	Apache-2.0	No
16	Claude Vision API (Anthropic)	Vision-Language LLM	2024	Commercial	No
17	GPT-4o / GPT-4.1 Vision (OpenAI)	Vision-Language LLM	2024	Commercial	No
18	Gemini 2.5 Vision (Google)	Vision-Language LLM	2025	Commercial (free tier)	No
19	InternVL2 / InternVL3	Vision-Language LLM	2024	MIT / Apache-2.0	No
20	Mask R-CNN (torchvision)	Pretrained CNN Detector	2017	BSD-style	Yes (unwired)
21	SAM 2 (Segment Anything)	Segmentation-only	2024	Apache-2.0	No

Detailed comparison — Part 1: capabilities (continued)

Section B — Detection capabilities

#	Model	Acc Photos	Acc Drawings	Custom Objects	Zero-shot
1	YOLOv11 (fine-tuned on floor pl...	95-98%	95-98%	Native (trained on your data)	No (but classes match you...
2	YOLOv11 (pretrained COCO)	85-95%	60-80%	Excellent — fine-tune with ~300 labels	No
3	YOLOv8 (pretrained COCO)	85-95%	60-80%	Excellent	No
4	YOLOv10 (pretrained)	85-93%	60-78%	Excellent	No
5	YOLO-NAS (Deci AI)	85-94%	60-80%	Good (via SuperGradients)	No
6	YOLOv5 (pretrained)	80-90%	60-75%	Good	No
7	YOLOv12 (pretrained)	88-96%	62-82%	Excellent	No
8	YOLOv9 (pretrained)	84-92%	58-76%	Good	No
9	RTMDet (OpenMMLab)	84-92%	60-78%	Excellent (via MMDetection)	No
10	RT-DETR	80-90%	70-85%	Good	No
11	Florence-2 (Microsoft)	85-93%	80-90%	Prompt-based + partial fine-tuning	Yes
12	YOLO-World	80-90%	65-80%	Native (prompt-based, no training)	Yes (native)
13	OpenCV Template Matching (curre...	30-50%	40-70%	Via templates only (brittle)	No
14	Grounding DINO	80-90%	75-85%	Native (prompt-based)	Yes (native, high quality)
15	OWLv2 (Google)	78-88%	70-82%	Native (prompt-based)	Yes
16	Claude Vision API (Anthropic)	90-95%	90-95%	Native (prompt-based, any category)	Yes
17	GPT-4o / GPT-4.1 Vision (OpenAI)	90-95%	85-95%	Native (prompt-based)	Yes
18	Gemini 2.5 Vision (Google)	88-93%	85-92%	Native (prompt-based)	Yes
19	InternVL2 / InternVL3	85-92%	80-90%	Prompt-based	Yes
20	Mask R-CNN (torchvision)	60-75%	65-80%	Hard (heavy training loop)	No
21	SAM 2 (Segment Anything)	N/A (segments only)	N/A (segments only)	N/A (doesn't classify)	Yes (for segmentation, no...

Detailed comparison — Part 2: performance & business

Section A — Performance & deployment

#	Model	CPU Speed	Size	RAM	Deployment
1	YOLOv11 (fine-tuned on floor ...	100-300 ms	6-22 MB	100-300 MB	Very easy (drop-in weights)
2	YOLOv11 (pretrained COCO)	100-300 ms	6-22 MB	100-300 MB	Very easy (pip install ultralytics)
3	YOLOv8 (pretrained COCO)	100-400 ms	6-25 MB	100-300 MB	Very easy
4	YOLOv10 (pretrained)	80-250 ms	6-25 MB	100-300 MB	Very easy
5	YOLO-NAS (Deci AI)	120-350 ms	18-100 MB	200-500 MB	Medium (SuperGradients library)
6	YOLOv5 (pretrained)	150-500 ms	14-88 MB	150-400 MB	Very easy
7	YOLOv12 (pretrained)	120-350 ms	8-25 MB	150-350 MB	Very easy
8	YOLOv9 (pretrained)	150-450 ms	10-50 MB	200-400 MB	Medium (less polished than Ultralytics)
9	RTMDet (OpenMMLab)	140-380 ms	20-100 MB	250-500 MB	Medium (MMDetection setup)
10	RT-DETR	200-800 ms	40-100 MB	500 MB - 1 GB	Medium
11	Florence-2 (Microsoft)	400-1500 ms	230-770 MB	1-3 GB	Medium
12	YOLO-World	300-800 ms	50-150 MB	500 MB	Medium
13	OpenCV Template Matching (cur...	100-500 ms	0 MB (rules only)	50 MB	Already deployed
14	Grounding DINO	800 ms - 3 s (CPU)	700 MB - 1.5 GB	2-4 GB	Medium (heavy deps)
15	OWLv2 (Google)	500-1500 ms	600 MB - 1.2 GB	2 GB	Medium
16	Claude Vision API (Anthropic)	1-3 seconds (network)	N/A (hosted)	0 (hosted)	Very easy (API key + HTTP)
17	GPT-4o / GPT-4.1 Vision (Open...	1-3 seconds (network)	N/A (hosted)	0 (hosted)	Very easy (API key + HTTP)
18	Gemini 2.5 Vision (Google)	1-4 seconds	N/A (hosted)	0 (hosted)	Very easy
19	InternVL2 / InternVL3	2-8 seconds	1-8 GB	4-20 GB	Hard (large model)
20	Mask R-CNN (torchvision)	2-10 seconds	180 MB	1.5-3 GB	Medium (heavy PyTorch)
21	SAM 2 (Segment Anything)	300 ms - 2 s	160-900 MB	1-3 GB	Medium

Detailed comparison — Part 2: performance & business (continued)

Section B — Integration & business

#	Model	Integration	Cost	Best Use	Limitations
1	YOLOv11 (fine-tuned on floor ...)	Easy (same code as pretrained)	Free (AGPL)	Best long-term option once you have data	Needs 300+ labeled floor plans ...
2	YOLOv11 (pretrained COCO)	Easy (~30 lines Python)	Free (AGPL restriction)	Best free ready-to-use detector	AGPL; no zero-shot for unseen classes
3	YOLOv8 (pretrained COCO)	Easy	Free (AGPL)	Rock-solid alternative to v11	Slightly older than v11
4	YOLOv10 (pretrained)	Easy	Free (AGPL)	When you need faster inference	AGPL; marginally less accurate
5	YOLO-NAS (Deci AI)	Medium	Free (Apache-2.0 = commercial)	When you need commercial-friendly	Smaller community than Ultralytics
6	YOLOv5 (pretrained)	Easy	Free (AGPL)	Legacy stability if v11 too new	Older, superseded by v8/v11
7	YOLOv12 (pretrained)	Easy	Free (AGPL)	For teams tracking the newest models	Too new for production certainty
8	YOLOv9 (pretrained)	Medium (~60 lines Python)	Free (more permissive than AGPL)	When AGPL is a legal blocker	Less polished ecosystem than Ultralytics
9	RTMDet (OpenMMLab)	Medium (~80 lines)	Free (Apache-2.0)	Commercial-friendly YOLOv11 alternative	YOLOv11 Detection framework has a steeper learning curve
10	RT-DETR	Medium	Free (Apache-2.0)	Transformer-first teams	Slower than YOLO on CPU
11	Florence-2 (Microsoft)	Medium	Free (MIT)	Open-source alternative to Gemini	Requires more RAM than YOLO
12	YOLO-World	Medium	Free (AGPL)	New class categories without retraining	Slower than YOLO; accuracy drops on new classes
13	OpenCV Template Matching (current)	Already integrated	Free	Baseline; useful for known objects	Can never generalize; accuracy drops on new classes
14	Grounding DINO	Medium	Free (Apache-2.0)	Best zero-shot accuracy if you have a GPU	Slow on CPU; large model
15	OWLv2 (Google)	Medium	Free (Apache-2.0)	Faster alternative to Grounding DINO	Slower than YOLO; smaller community
16	Claude Vision API (Anthropic)	Very easy (~50 lines)	Paid (~\$0.01-0.05 per image)	Fastest path to production	Costs per call; requires internet
17	GPT-4o / GPT-4.1 Vision (OpenAI)	Very easy (~50 lines)	Paid (~\$0.01-0.05 per image)	Equivalent to Claude Vision	Costs per call; requires internet
18	Gemini 2.5 Vision (Google)	Very easy	Free tier + paid	Cheapest hosted VLM (has free tier)	Free tier has rate limits; slightly slower
19	InternVL2 / InternVL3	Medium	Free (open weights)	Open-source competitor to Gemini	Needs a GPU with ≥16 GB VRAM for full resolution
20	Mask R-CNN (torchvision)	Medium	Free	Only if already wired in	9 years old; inferior to YOLOv11
21	SAM 2 (Segment Anything)	High (needs classifier on top)	Free (Apache-2.0)	Interactive editors (click-to-segment)	Does NOT identify chairs vs tables

Technical scores (0-10 per dimension, weighted overall)

Acc = Accuracy · Spd = Speed · Cust = Custom training · Depl = Deployment · Prod = Production readiness · Cost = Cost score · Eff = Efficiency

#	Model	Acc	Spd	Cust	Depl	Prod	Cost	Eff	OVERALL
1	YOLOv11 (fine-tuned on floor plan...	9.5	8.0	10.0	9.0	10.0	8.0	10.0	9.19
2	YOLOv11 (pretrained COCO)	8.0	8.0	9.0	9.0	10.0	8.0	10.0	8.66
3	YOLOv8 (pretrained COCO)	8.0	8.0	9.0	9.0	10.0	8.0	10.0	8.66
4	YOLOv10 (pretrained)	8.0	9.0	9.0	8.5	8.0	8.0	10.0	8.49
5	YOLO-NAS (Deci AI)	8.0	8.0	8.0	7.0	8.0	10.0	8.0	8.09
6	YOLOv5 (pretrained)	7.0	7.5	8.0	9.0	10.0	8.0	8.0	8.06
7	YOLOv12 (pretrained)	8.5	7.5	9.0	8.0	6.0	8.0	9.0	8.02
8	YOLOv9 (pretrained)	7.5	7.5	8.0	7.0	8.0	10.0	9.0	7.95
9	RTMDet (OpenMMLab)	8.0	7.5	8.0	6.0	8.0	10.0	8.0	7.87
10	RT-DETR	8.0	6.5	7.5	6.0	7.0	10.0	6.5	7.43
11	Florence-2 (Microsoft)	8.5	6.0	6.5	6.0	7.5	10.0	6.5	7.39
12	YOLO-World	7.5	6.5	7.5	6.5	7.0	8.0	7.5	7.20
13	OpenCV Template Matching (current)	4.5	8.0	3.0	10.0	8.0	10.0	10.0	7.04
14	Grounding DINO	8.0	3.5	8.0	5.5	7.5	10.0	4.0	6.89
15	OWLv2 (Google)	7.5	5.0	7.0	5.0	7.0	10.0	5.0	6.76
16	Claude Vision API (Anthropic)	9.0	4.0	3.0	9.0	9.5	4.0	8.0	6.75
17	GPT-4o / GPT-4.1 Vision (OpenAI)	9.0	4.0	3.0	9.0	9.5	4.0	8.0	6.75
18	Gemini 2.5 Vision (Google)	8.5	3.5	3.0	9.0	8.5	6.0	8.0	6.67
19	InternVL2 / InternVL3	8.5	3.0	5.0	4.0	7.0	10.0	3.0	6.15
20	Mask R-CNN (torchvision)	6.5	3.0	6.0	5.0	8.0	10.0	3.0	6.07
21	SAM 2 (Segment Anything)	4.0	5.0	4.0	5.0	8.0	10.0	4.0	5.50

Per-model recommendation — verdict

Every model's suitability and priority.

#	Model	Overall	Suitable?	Priority	Why Recommended
1	YOLOv11 (fine-tuned on floor ...	9.19	Yes (Phase 2 goal)	★★★★★	Best long-term option once you have labeled data
2	YOLOv11 (pretrained COCO)	8.66	Yes	★★★★★	Best free ready-to-use detector for THIS project
3	YOLOv8 (pretrained COCO)	8.66	Yes	★★★★	Rock-solid alternative to v11
4	YOLOv10 (pretrained)	8.49	Yes	★★★★	When you need faster inference than v11
5	YOLO-NAS (Deci AI)	8.09	Yes (if commercial ...	★★★★	When you need commercial-friendly YOLO
6	YOLOv5 (pretrained)	8.06	Yes (but not first ...	★★★	Legacy stability if v11 tooling breaks
7	YOLOv12 (pretrained)	8.02	Yes (but wait 6-12 ...	★★★	For teams tracking the newest release
8	YOLOv9 (pretrained)	7.95	Yes (if AGPL matter...	★★★★★	When AGPL is a legal blocker
9	RTMDet (OpenMMLab)	7.87	Yes (if okay with M...	★★★	Commercial-friendly YOLO alternative
10	RT-DETR	7.43	Yes (niche)	★★★	Transformer-first teams
11	Florence-2 (Microsoft)	7.39	Yes (great mid-tier...	★★★★★	Open-source alternative to Claude/GPT vision — runs lo...
12	YOLO-World	7.20	Yes (niche)	★★★	New class categories without retraining
13	OpenCV Template Matching (cur...	7.04	Only as fallback	★★	Baseline; useful for known chair styles
14	Grounding DINO	6.89	Yes (if GPU availab...	★★★	Best zero-shot accuracy if you have a GPU
15	OWLv2 (Google)	6.76	Yes (niche)	★★★	Faster alternative to Grounding DINO
16	Claude Vision API (Anthropic)	6.75	Yes (if paid is acc...	★★★★★	Fastest path to production-grade accuracy
17	GPT-4o / GPT-4.1 Vision (Open...	6.75	Yes (if paid is acc...	★★★★★	Equivalent to Claude Vision in most tasks
18	Gemini 2.5 Vision (Google)	6.67	Yes (for low volume)	★★★★★	Cheapest hosted VLM (has free tier)
19	InternVL2 / InternVL3	6.15	No (too heavy for t...	★★	Open-source competitor to Claude/GPT for local deploym...
20	Mask R-CNN (torchvision)	6.07	No (use YOLOv11 ins...	★	Only if already wired in
21	SAM 2 (Segment Anything)	5.50	No (wrong category ...	★★	Interactive editors (click-to-segment); NOT a full det...

Per-model recommendation — pros & cons

Continuation: advantages and disadvantages of each option.

#	Model	Advantages	Disadvantages
1	YOLOv11 (fine-tuned on floor ...	Near-perfect accuracy on YOUR floor plans	Upfront labeling cost; retraining needed if domain shi...
2	YOLOv11 (pretrained COCO)	Huge community; tiny model; easy fine-tune; CPU-fine...	AGPL license friction; needs training for new classes
3	YOLOv8 (pretrained COCO)	Most battle-tested Ultralytics YOLO in production	v11 exists — pick that unless you have reason
4	YOLOv10 (pretrained)	NMS-free = simpler pipeline; slightly faster	Small accuracy dip vs v11
5	YOLO-NAS (Deci AI)	Apache-2.0 = redistribute freely; NAS-optimized archite...	SuperGradients library less polished than Ultralytics
6	YOLOv5 (pretrained)	Enormous ecosystem, still deployed everywhere	Older architecture; use v8/v11 for new work
7	YOLOv12 (pretrained)	Newest, uses attention, slightly higher accuracy	Less field-tested; some tooling not caught up
8	YOLOv9 (pretrained)	More permissive license than Ultralytics	Smaller community; less tooling
9	RTMDet (OpenMMLab)	Apache-2.0; benchmark-competitive with YOLO	MMDetection = heavier framework than Ultralytics
10	RT-DETR	Handles cluttered scenes better; Apache license	Slower; smaller community than YOLO
11	Florence-2 (Microsoft)	MIT license; runs locally; compact multimodal	Slower than YOLO; less mature ecosystem than Ultralyti...
12	YOLO-World	Detect new classes via text prompt without training	Slower and less accurate than fine-tuned YOLO
13	OpenCV Template Matching (cur...	Zero deps; already working; instant	Fundamental accuracy cap; brittle to new styles
14	Grounding DINO	Best zero-shot; Apache-2.0; text prompts	Very slow on CPU; heavy
15	OWLv2 (Google)	Google research; Apache-2.0; text prompts	Fewer integrations than Grounding DINO
16	Claude Vision API (Anthropic)	Best 'just works' accuracy; near-zero setup; understand...	Recurring cost; needs internet; no coordinate guarante...
17	GPT-4o / GPT-4.1 Vision (Open...	Industry standard; huge ecosystem; JSON mode	Recurring cost; needs internet
18	Gemini 2.5 Vision (Google)	Free tier available; good for low-volume	Rate-limited free tier; slightly less accurate
19	InternVL2 / InternVL3	MIT-licensed open VLM; comparable to GPT-4o on many ta...	Heavy; needs GPU; not fit for this project's CPU box
20	Mask R-CNN (torchvision)	Already in the project; produces masks + boxes	Slow; heavy; outdated
21	SAM 2 (Segment Anything)	Perfect segmentation of any shape	Not a detector — you need to pair with a classifier